

```
#!/usr/bin/env python3
"""
Fetch current USD exchange rates and save a
timestamped snapshot.

Data source: https://www.exchangerate-
api.com/docs/free (Open Access, no API key)
Endpoint used: https://open.er-
api.com/v6/latest/USD

Behavior:
- Fetches rates for USD -> [EUR, GBP, ETB,
+ 7 more currencies]
- Saves each run as data/rates_<UTC
timestamp>.json
- Also appends a row to
data/rates_history.csv (easy to load with
pandas)
- Stops collecting once 12 snapshots exist
(i.e. after ~12 hourly runs),
  so the workflow can keep running on
  schedule without growing forever.
"""

import csv
import json
import os
import sys
from datetime import datetime, timezone
```

```

import requests

API_URL = "https://open.er-
api.com/v6/latest/USD"

# 3 required currencies + 7 of choice = 10
total
CURRENCIES = [
    "EUR", # Euro
    "GBP", # British Pound
    "ETB", # Ethiopian Birr
    "JPY", # Japanese Yen
    "CAD", # Canadian Dollar
    "AUD", # Australian Dollar
    "CHF", # Swiss Franc
    "CNY", # Chinese Yuan
    "INR", # Indian Rupee
    "KES", # Kenyan Shilling
]

DATA_DIR =
os.path.join(os.path.dirname(os.path.abspath(
__file__)), "data")
HISTORY_CSV = os.path.join(DATA_DIR,
"rates_history.csv")
MAX_SNAPSHOTS = 12 # run for "up to 12
hours" at 1 snapshot/hour

def count_existing_snapshots() -> int:
    if not os.path.isdir(DATA_DIR):
        return 0
    return len(
        [f for f in os.listdir(DATA_DIR) if

```

```

f.startswith("rates_") and
f.endswith(".json")]
    )

def fetch_rates() -> dict:
    resp = requests.get(API_URL,
        timeout=20)
    resp.raise_for_status()
    payload = resp.json()
    if payload.get("result") != "success":
        raise RuntimeError(f"API did not
return success: {payload}")
    return payload["rates"]

def save_snapshot(rates: dict, timestamp:
datetime) -> str:
    os.makedirs(DATA_DIR, exist_ok=True)
    ts_str =
timestamp.strftime("%Y%m%dT%H%M%S")
    snapshot = {
        "timestamp_utc":
timestamp.isoformat(),
        "base": "USD",
        "rates": {cur: rates.get(cur) for
cur in CURRENCIES},
    }

    json_path = os.path.join(DATA_DIR,
f"rates_{ts_str}.json")
    with open(json_path, "w") as f:
        json.dump(snapshot, f, indent=2)

```

```

        # Append to a single CSV history file
        for easy pandas loading
        file_exists =
os.path.isfile(HISTORY_CSV)
        with open(HISTORY_CSV, "a", newline="")
as f:
            writer = csv.writer(f)
            if not file_exists:

writer.writerow(["timestamp_utc"] +
CURRENTCIES)

writer.writerow([snapshot["timestamp_utc"]]
+ [snapshot["rates"][c] for c in
CURRENTCIES])

        return json_path

def main():
    existing = count_existing_snapshots()
    if existing >= MAX_SNAPSHOTS:
        print(
            f"Already have {existing}
snapshots (limit {MAX_SNAPSHOTS}). "
            "Skipping fetch – 12-hour
collection window is complete."
        )
        return 0

    now = datetime.now(timezone.utc)
    try:
        rates = fetch_rates()
    except Exception as exc:

```

```
        print(f"Error fetching rates:
{exc}", file=sys.stderr)
        return 1

    path = save_snapshot(rates, now)
    print(f"Saved snapshot {existing +
1}/{MAX_SNAPSHOTS} -> {path}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```